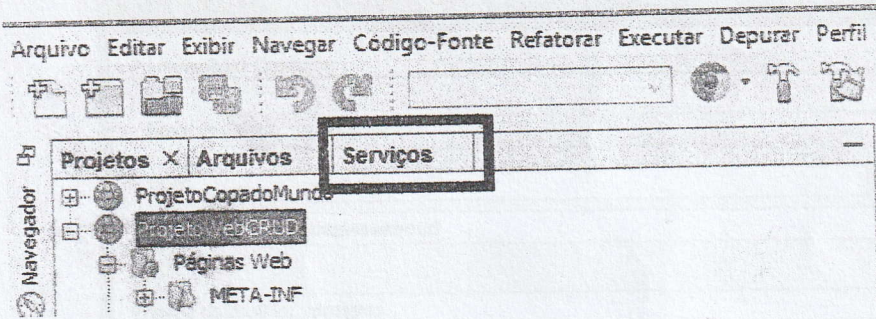


arquivo > Novo Projeto > Java Web > Aplicações Web

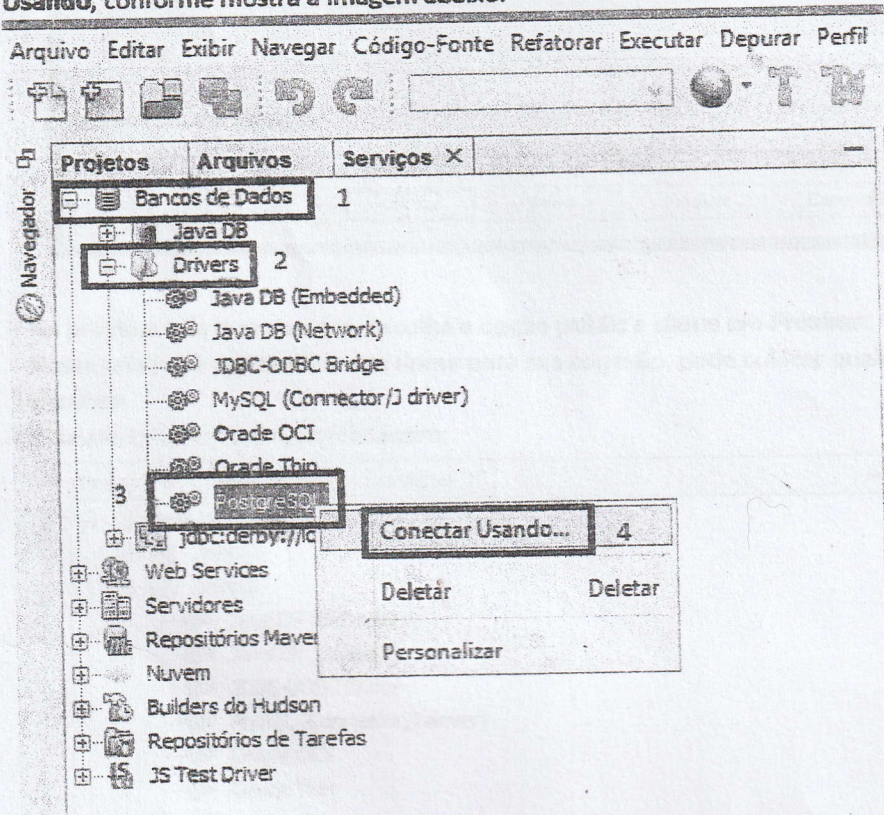
PASSO A PASSO PARA CRIAR UM PROJETO EM JAVA

1. Depois de ter criado o banco de dados, devemos criar uma conexão com o Netbeans.
2. Clique na aba **Serviços**



Prof. André Murato

3. Em seguida clique em **Banco de Dados > Drivers**
4. Clique com o botão direito do mouse em cima da opção **PostgreSQL** e escolha a opção **Conectar** Usando, conforme mostra a imagem abaixo.

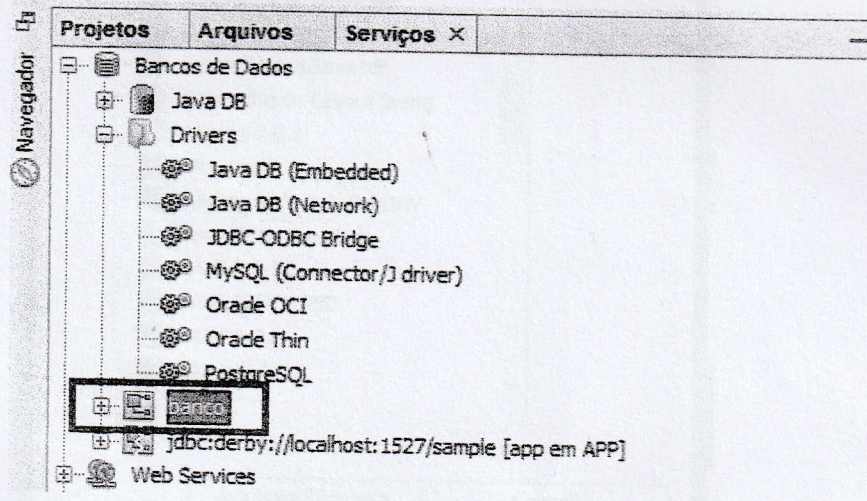


model > DAO > View > Controller

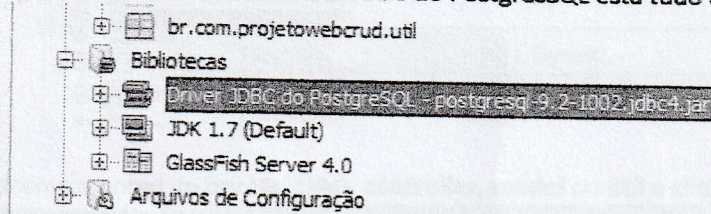
Shift + Alt + F

5. Na tela irá aparecer preencha os campos conforme mostra a imagem, lembrando que você deve alterar o nome do banco de dados e sempre clicar no botão **Testar Conexão**, para poder verificar se realmente está tudo correto. Feito isso, clique no botão **Próximo**.

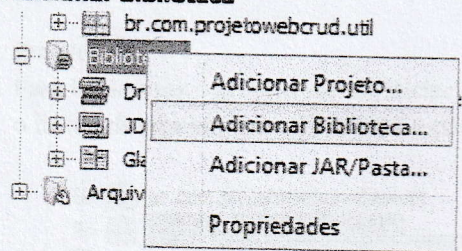
6. Na próxima tela que aparece, escolha a opção **public** e clique em **Próximo**.
7. Nessa tela você vai escolher um nome para sua conexão, pode colocar qualquer nome e clicar em **Finalizar**.
8. Essa conexão deverá aparecer assim:



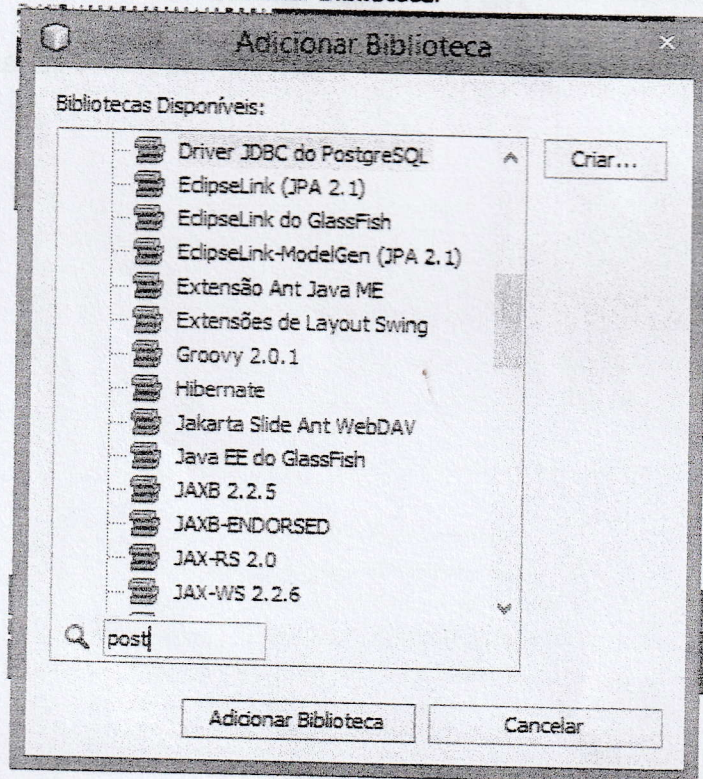
9. Feito isso, vamos verificar se nas **Bibliotecas** do projeto está o driver do Postgres como mostra a imagem abaixo. Se estiver Driver JDBC do PostgreSQL está tudo certo, senão siga os passos abaixo.



10. Clique com o botão direito do mouse em cima de **Bibliotecas** e em seguida escolha a opção **Adicionar Biblioteca**

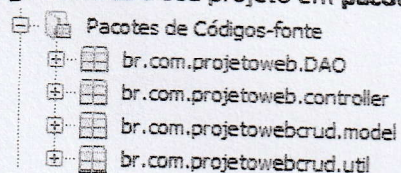


11. Na tela que surge comece a digitar a palavra **post** e escolha a opção **Driver JDBC do PostgreSQL** e clique no botão **Adicionar Biblioteca**.

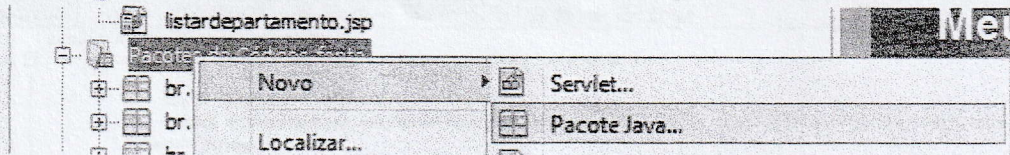


12. Pronto!

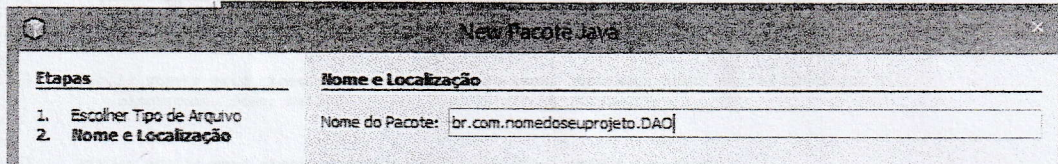
13. Agora divida o seu projeto em **pacotes** para facilitar na programação



14. Lembrando que o nome **projetoweb** é só um exemplo, substitua esse nome pelo nome do seu projeto, caso não saiba criar um pacote, clique com o botão direito em cima de **Pacotes de Códigos-fonte > Novo > Pacote Java...**

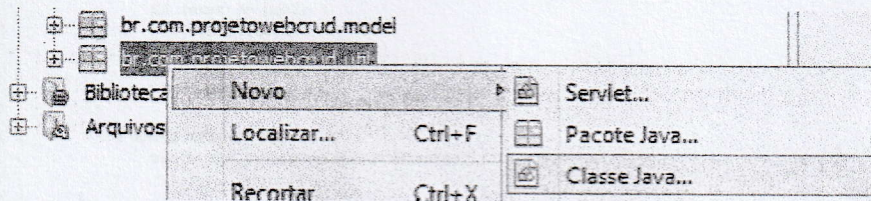


15. Informe o nome do pacote, **DAO, controller, model** ou **útil** e clique no botão **Finalizar**



16. Pronto!

17. Para iniciarmos o projeto vamos iniciar pelo **pacote útil**, criando um **ConnectionFactory**, clique com o botão direito em cima do pacote e crie uma **Classe Java**



18. Dê o nome para essa Classe Java de: **ConnectionFactory**

19. Implemente o código abaixo:

```

ConnectionFactory.java
Código-Fonte  Histórico
17 public class ConnectionFactory {
18     public static Connection getConnection() throws Exception {
19         try {
20             Class.forName("org.postgresql.Driver");
21             return DriverManager.getConnection("jdbc:postgresql://localhost:5432/bancoprojetoweb","postgres","123456");
22         } catch (Exception ex) {
23             throw new Exception(ex.getMessage());
24         }
25     }
26     public static void closeConnection(Connection conn, Statement stmt, ResultSet rs) throws Exception {
27         close(conn, stmt, rs);
28     }
29
30     public static void closeConnection(Connection conn, Statement stmt) throws Exception {
31         close(conn, stmt, null);
32     }
33
34     public static void closeConnection(Connection conn) throws Exception {
35         close(conn, null, null);
36     }
37
38     private static void close(Connection conn, Statement stmt, ResultSet rs) throws Exception {
39         try {
40             if (rs != null) {
41                 rs.close();
42             }
43             if (stmt != null) {
44                 stmt.close();
45             }
46             if (conn != null) {
47                 conn.close();
48             }
49         } catch (Exception ex) {
50             throw new Exception(ex.getMessage());
51         }
52     }
53
54     public static void close(Connection conn, PreparedStatement stmt) {
55         throw new UnsupportedOperationException("Not supported yet."); //To change body of generated methods, choose Tools
56     }
57 }

```

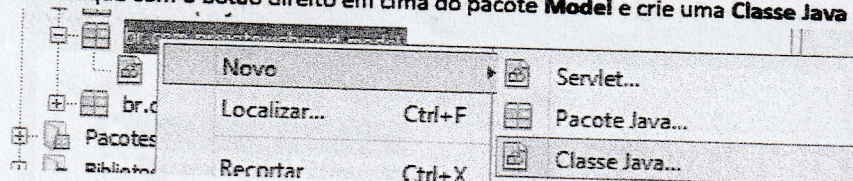
20. Verifique se as importações estão corretas.

```

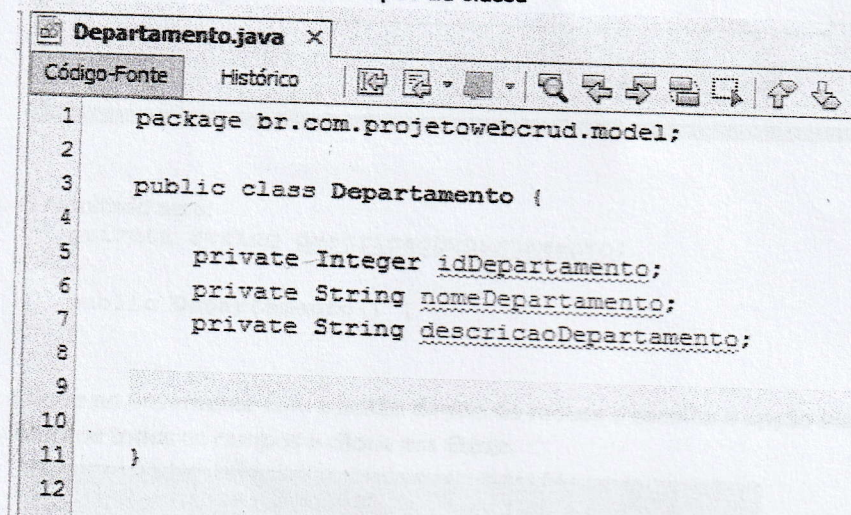
ConnectionFactory.java
Código-Fonte  Histórico
1  /*
2   * To change this template, choose Tools | Templates
3   * and open the template in the editor.
4   */
5   package br.com.projetoowebcrud.util;
6
7   import java.sql.Statement;
8   import java.sql.Connection;
9   import java.sql.DriverManager;
10  import java.sql.PreparedStatement;
11  import java.sql.ResultSet;
12
13  /**
14   *
15   * @author Igor Paulino
16   */
17  public class ConnectionFactory {
18      public static Connection getConnection() throws Exception {

```

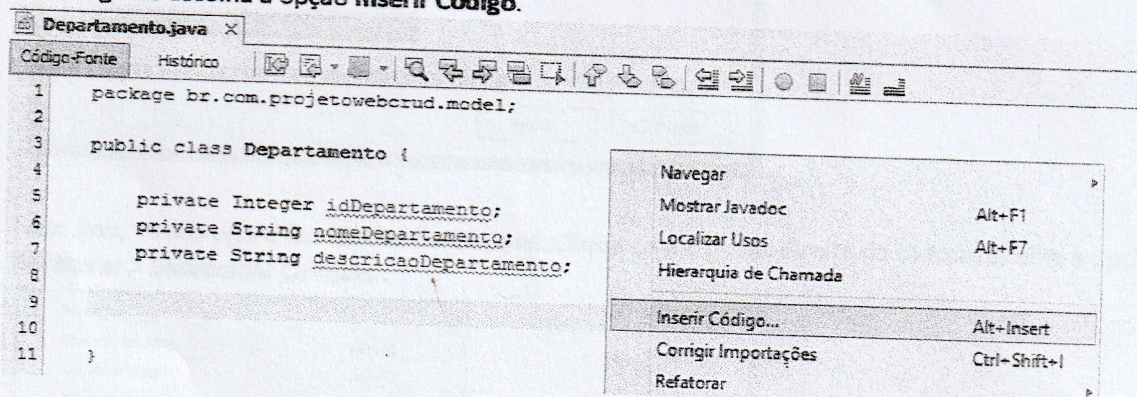
21. Feito a **ConnectionFactory** vamos agora implementar as Classes do nosso projeto para poder gerar os Gets e Sets. Clique com o botão direito em cima do pacote **Model** e crie uma **Classe Java**



22. Com a classe criada, crie os campos da classe



23. Atributos da classe criado, agora vamos inserir os **Construtores**. Clique com o botão direito do mouse e em seguida escolha a opção **Inserir Código**.

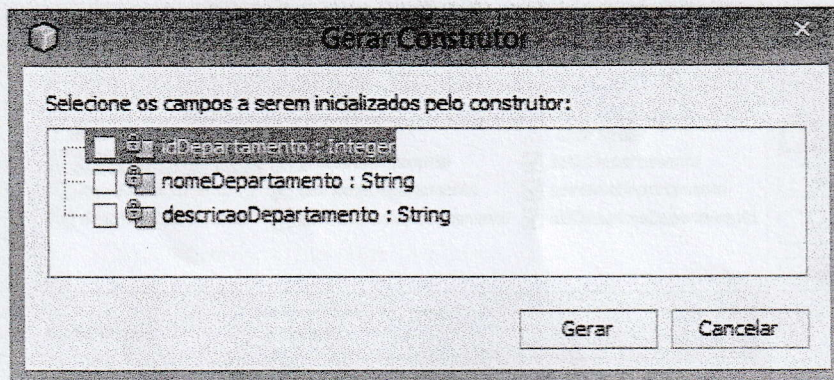


24. Escolha a opção **Construtor...**

```
private String nomeDepartamento;
private String descricaoDepartamento;
```



25. Não marque nenhuma opção e clique em Gerar.



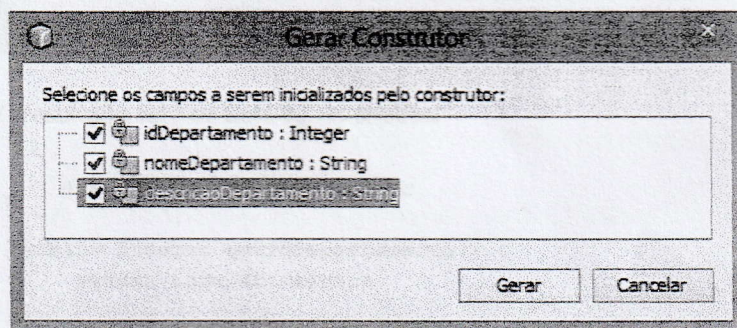
OBS: Quando for fazer
1º -> Sem marcar
2º -> marcando a 2ª opção

26. O resultado será:

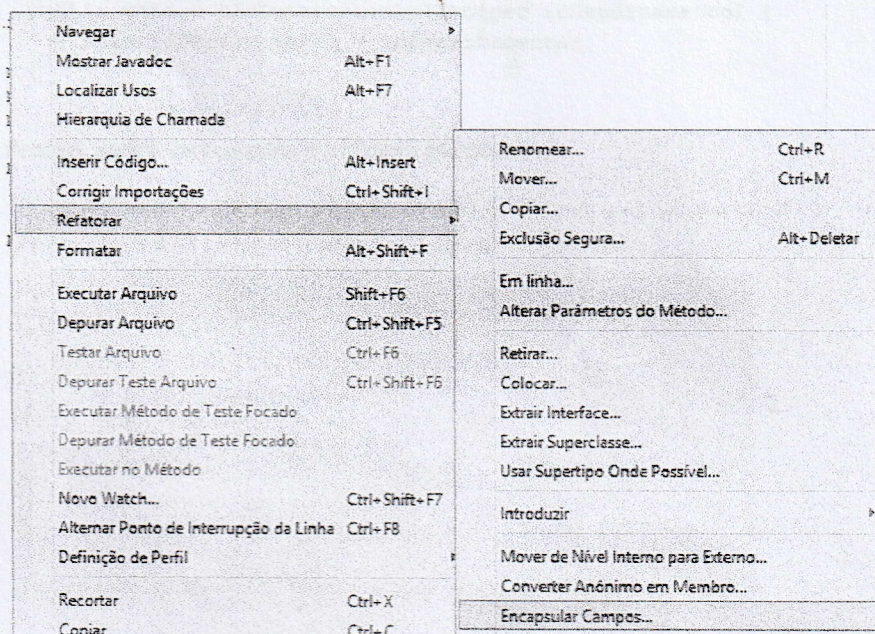
```
private String descricaoDepartamento;  
  
public Departamento() {  
}
```

27. Clique novamente com o botão direito do mouse e escolha a opção Inserir Código > Construtor...

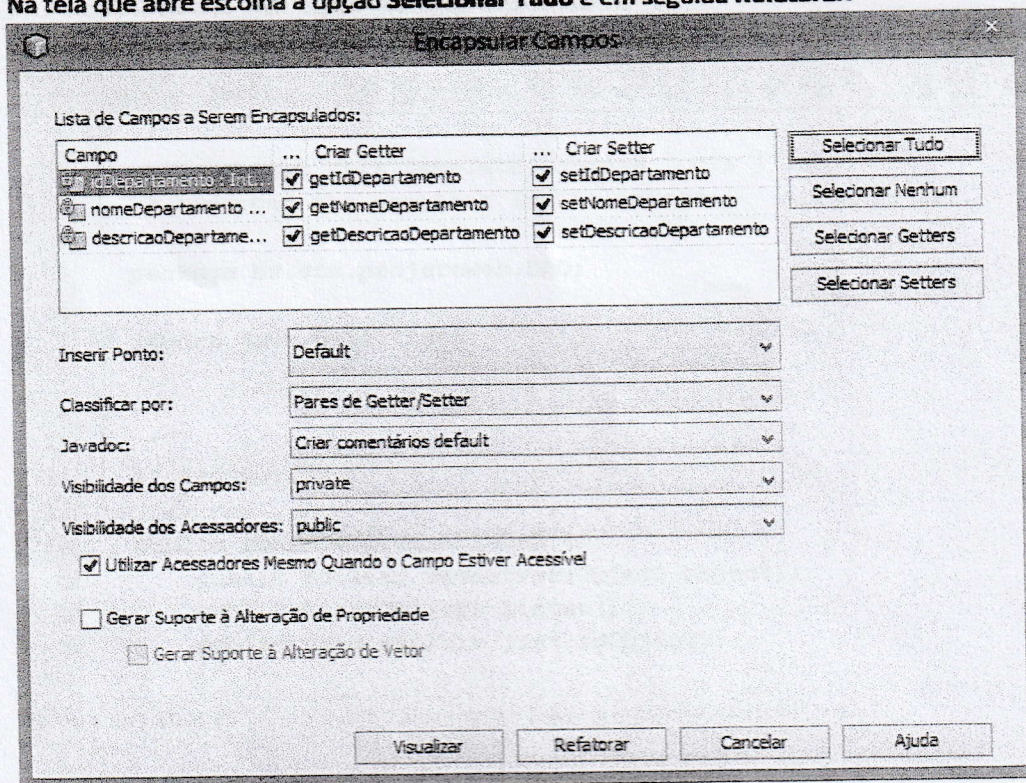
28. Marque todos os campos e clique em Gerar.



29. Feito isso, vamos agora Encapsular os Campos. Clique com o botão direito do mouse escolha a opção Refatorar > Encapsular Campos..



30. Na tela que abre escolha a opção **Selecionar Tudo** e em seguida **Refatorar**.



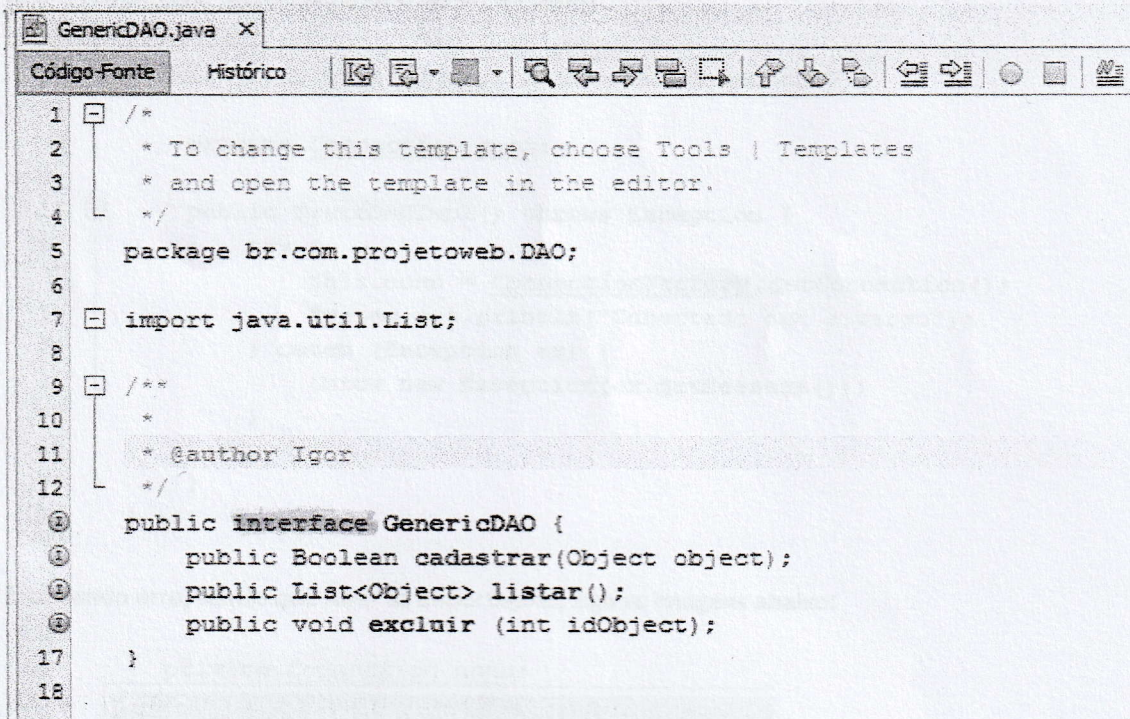
31. O resultado será os Gets e Sets criados.

```
/**
 * @return the idDepartamento
 */
public Integer getIdDepartamento() {
    return idDepartamento;
}

/**
 * @param idDepartamento the idDepartamento to set
 */
public void setIdDepartamento(Integer idDepartamento) {
    this.idDepartamento = idDepartamento;
}
```

32. Pronto, agora vamos para o próximo pacote.

33. No Pacote DAO crie uma classe chamada GenericDAO.



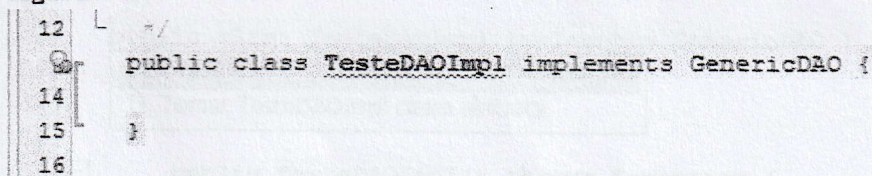
```
GenericDAO.java X
Código-Fonte  Histórico
1  /*
2   * To change this template, choose Tools | Templates
3   * and open the template in the editor.
4   */
5   package br.com.projetoweb.DAO;
6
7   import java.util.List;
8
9   /**
10    *
11    * @author Igor
12    */
13   public interface GenericDAO {
14       public Boolean cadastrar(Object object);
15       public List<Object> listar();
16       public void excluir (int idObject);
17   }
18
```

34. Não se esqueça de fazer a importação do `java.util.List`.

35. Pronto!

36. Agora crie uma classe chamada `TesteDAOImpl`, lembrando que `Teste` é o nome da classe que está sendo utilizado neste exemplo.

37. Depois de criar a classe adicione o seguinte texto: `implements GenericDAO`, conforme a imagem a seguir:



```
12  /*
13   *
14   * @author Igor
15   */
16   public class TesteDAOImpl implements GenericDAO {
17
18   }
```

38. Agora vamos fazer a conexão com o banco de dados.

```
11  L  */
12  public class TesteDAOImpl implements GenericDAO {
13
14      private Connection conn;
15
16      public TesteDAOImpl() throws Exception {
17          try {
18              this.conn = ConnectionFactory.getConnection();
19              System.out.println("Conectado com sucesso");
20          } catch (Exception ex) {
21              throw new Exception(ex.getMessage());
22          }
23      }
24  }
25  }
```

39. Está dando erro, temos que fazer as Importações, siga as imagens abaixo:

```
13  private Connection conn;
15  try {
16      this.conn = ConnectionFactory.getConnection();
21  }
```

Adiciona importação para java.sql.Connection

Adiciona importação para br.com.projetowebcrud.util.ConnectionFactory

40. Observe que tem um alerta na linha, clique nele e escolha a opção para Implementar Todos os Métodos Abstratos

```
14  L  */
15  public class TesteDAOImpl implements GenericDAO {
16
17      Implementa Todos os Métodos Abstratos
18      Tornar TesteDAOImpl classe abstrata
19
20  public TesteDAOImpl() throws Exception {
```

41. A sua classe ficará assim:

```

Código-Fonte  Histórico
public class TesteDAOImpl implements GenericDAO {
17
18     private Connection conn;
19
20     public TesteDAOImpl() throws Exception {
21         try {
22             this.conn = ConnectionFactory.getConnection();
23             System.out.println("Conectado com sucesso");
24         } catch (Exception ex) {
25             throw new Exception(ex.getMessage());
26         }
27     }
28
29     @Override
30     public Boolean cadastrar(Object object) {
31         throw new UnsupportedOperationException("Not supported yet."); //To change body of implementation, use IDE.
32     }
33
34     @Override
35     public List<Object> listar() {
36         throw new UnsupportedOperationException("Not supported yet."); //To change body of implementation, use IDE.
37     }
38
39     @Override
40     public void excluir(int idObject) {
41         throw new UnsupportedOperationException("Not supported yet."); //To change body of implementation, use IDE.
42     }
43
44 }
45

```

42. Apague esta linha dos métodos:

```
throw new UnsupportedOperationException("Not supported yet."); //To change body of implementation, use IDE.
```

43. Agora vamos implementar o Cadastrar:

```

@Override
public Boolean cadastrar(Object object) {
    Departamento departamento = (Departamento) object;
    PreparedStatement stmt = null;
    String sql = "INSERT INTO departamento(nome_departamento, descricao_departamento) VALUES (?, ?)";
    try {
        stmt = conn.prepareStatement(sql);
        stmt.setString(1, departamento.getNomeDepartamento());
        stmt.setString(2, departamento.getDescricaoDepartamento());
        stmt.executeUpdate();
        return true;
    } catch (SQLException ex) {
        System.out.println("Problemas ao cadastrar Departamento! Erro: " + ex.getMessage());
        ex.printStackTrace();
        return false;
    } finally {
        try {
            ConnectionFactory.closeConnection(conn, stmt);
        } catch (Exception ex) {
            System.out.println("Problemas ao fechar os parâmetros de conexão! Erro: " + ex.getMessage());
            ex.printStackTrace();
        }
    }
}

```

44. Lembre-se sempre de trocar os nomes

45. Agora o Listar

```
@Override
public List<Object> listar() {
    List<Object> resultado = new ArrayList<Object>();
    PreparedStatement stmt = null;
    ResultSet rs = null;
    String sql = "select d.* from departamento d;";
    try {
        stmt = conn.prepareStatement(sql);
        rs = stmt.executeQuery();
        while (rs.next()) {
            Departamento departamento = new Departamento();
            departamento.setIdDepartamento(rs.getInt("id_departamento"));
            departamento.setNomeDepartamento(rs.getString("nome_departamento"));
            departamento.setDescricaoDepartamento(rs.getString("descricao_departamento"));
            resultado.add(departamento);
        }
    } catch (SQLException ex) {
        System.out.println("Problemas ao listar departamentos! Erro: " + ex.getMessage());
        ex.printStackTrace();
    } finally {
        try {
            ConnectionFactory.closeConnection(conn, stmt, rs);
        } catch (Exception ex) {
            System.out.println("Problemas ao fechar os parâmetros de conexão! Erro: " + ex.getMessage());
            ex.printStackTrace();
        }
    }
    return resultado;
}
```

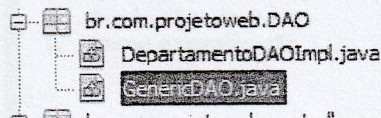
46. Agora o Excluir

```
89 @Override
90 public void excluir(int idObject) {
91     PreparedStatement stmt = null;
92     String sql = "delete from departamento where id_departamento = ?;";
93     try {
94         stmt = conn.prepareStatement(sql);
95         stmt.setInt(1, idObject);
96         stmt.executeUpdate();
97     } catch (SQLException ex) {
98         System.out.println("Problemas na DAO ao excluir Departamento! Erro: " + ex.getMessage());
99         ex.printStackTrace();
100     } finally {
101         try {
102             ConnectionFactory.closeConnection(conn, stmt);
103         } catch (Exception ex) {
104             System.out.println("Problemas ao fechar os parâmetros de conexão! Erro: " + ex.getMessage());
105             ex.printStackTrace();
106         }
107     }
108 }
109 }
```

47. Caso apareça algum erro, você pode verificar as importações para verificar se está corretas:

```
DepartamentoDAOImpl.java x
Código-Fonte Histórico
1 /*
2  * To change this template, choose Tools | Templates
3  * and open the template in the editor.
4  */
5 package br.com.projetoweb.DAO;
6
7 import br.com.projetowebcrud.model.Departamento;
8 import br.com.projetowebcrud.util.ConnectionFactory;
9 import java.sql.Connection;
10 import java.sql.PreparedStatement;
11 import java.sql.ResultSet;
12 import java.sql.SQLException;
13 import java.util.ArrayList;
14 import java.util.List;
15
16 /**
```

48. O nosso pacote DAO tem que estar assim:



49. Implementamos a DAO, agora iremos implementar a View e a Controller

50. Abra a página index de seu projeto e crie os links necessários de Cadastro e o de Listar, lembrando que o Listar não chamará uma página JSP e sim um Servlet.

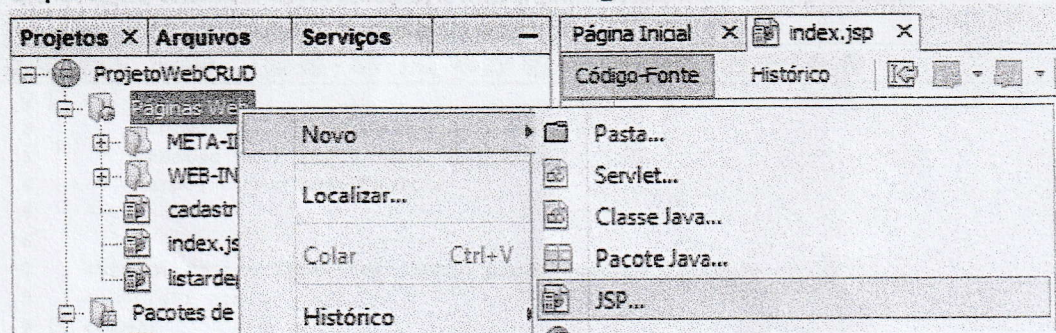
51. Sua index.jsp deverá estar assim:

```
Página Inicial x index.jsp x
Código-Fonte Histórico
1 <!--
2     Document    : index
3     Created on  : 24/02/2014, 22:20:07
4     Author      : Igor Paulino
5 -->
6
7 <%@page contentType="text/html" pageEncoding="UTF-8"%>
8 <!DOCTYPE html>
9 <html>
10 <head>
11     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
12     <title>JSP Page</title>
13 </head>
14 <body>
15     <h1>Hello World!</h1>
16     <a href="cadastrardepartamento.jsp">Cadastrar Departamento </a><br />
17     <a href="ListarDepartamento">Listar Departamento </a>
18 </body>
19 </html>
20
```

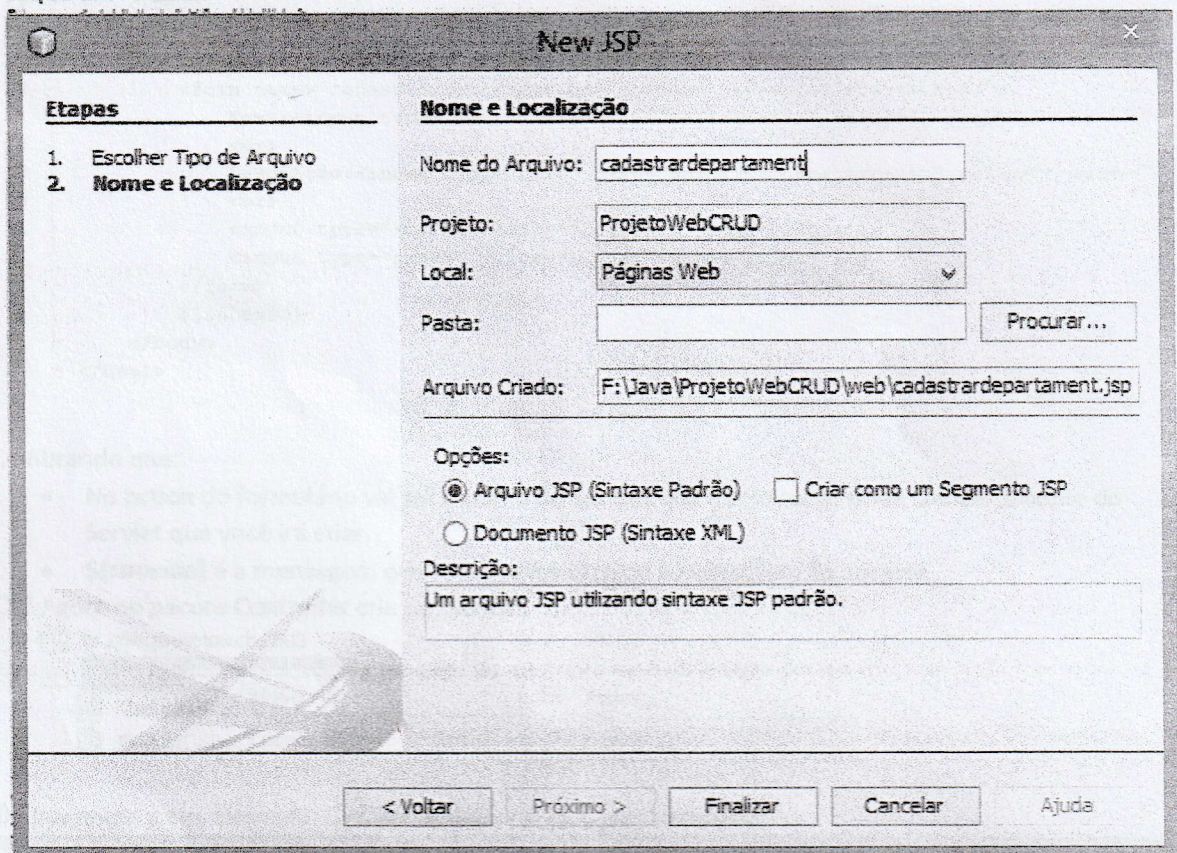
52. Não se esqueça de alterar os nome

53. Feito isso iremos criar uma página JSP.

54. Clique com o botão direito do mouse em cima de **Páginas Web > Novo > JSP**.



55. Informe um nome para a página no caso do exemplo será cadastrar departamento, tudo minúsculo e clique em Finalizar.



56. Abra a página cadastrar departamento e implemente conforme o exemplo abaixo:

```

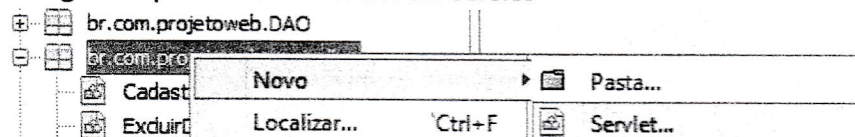
Página Inicial x index.jsp x cadastrardepartamento.jsp x
Código-Fonte Histórico
1 <!--
2 Document : cadastrardepartamento.jsp
3 Created on : 18/03/2014, 22:12:51
4 Author : Igor Paulino
5 -->
6
7 <%@page contentType="text/html" pageEncoding="UTF-8"%>
8 <!DOCTYPE html>
9 <html>
10 <head>
11 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
12 <title>Cadastro de Departamento</title>
13 </head>
14 <body>
15 <h1>Cadastro de Departamento!</h1>
16 <form name="cadastrardepartamento" action="CadastrarDepartamento">
17 Nome:<input type="text" name="nomeDepartamento" value="" size="15" />
18 <br>
19 Descrição:<input type="text" name="descricaoDepartamento" value="" size="30" />
20 <br>
21 <input type="submit" value="Cadastrar" name="cadastrar" />
22 <input type="reset" value="Limpar" name="limpar" />
23 </form>
24 ${sucesso}
25 </body>
26 </html>
27

```

57. Lembrando que:

- No **action** do formulário vai ser o nome do **Servlet** que iremos criar pode colocar o nome do Servlet que você irá criar.
- **\${sucesso}** é a mensagem que retornará o erro ou a mensagem de sucesso

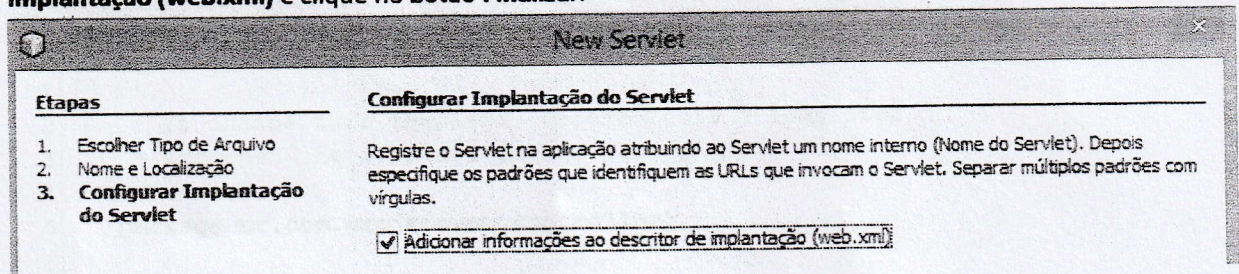
58. Ok! Agora no pacote **Controller** crie um **Servlet**



59. Dê um nome a ele. Exemplo: CadastrarFuncionario, CadastrarAluno.

New Servlet	
Etapas	Nome e Localização
1. Escolher Tipo de Arquivo	Nome da Classe: NomeDoServlet
2. Nome e Localização	
3. Configurar Implantação do Servlet	Projeto: ProjetoWebCRUD

60. Clique no botão Próximo e logo em seguida marque a opção **Adicionar informações ao descritor de implantação (web.xml)** e clique no botão **Finalizar**.



61. O Servlet está criado, agora implemente o código abaixo. Não se esqueça de alterar os nomes.

```
CadastrarDepartamento.java x
Código-Fonte  Histórico
19  * @author Igor Paulino
20  */
21  public class CadastrarDepartamento extends HttpServlet {
22
23      /**
24       * Processes requests for both HTTP
25       * <code>GET</code> and
26       * <code>POST</code> methods.
27       *
28       * @param request servlet request
29       * @param response servlet response
30       * @throws ServletException if a servlet-specific error occurs
31       * @throws IOException if an I/O error occurs
32       */
33      protected void processRequest(HttpServletRequest request, HttpServletResponse response)
34          throws ServletException, IOException {
35          String nomeDepartamento = request.getParameter("nomeDepartamento");
36          String descricaoDepartamento = request.getParameter("descricaoDepartamento");
37          String mensagem = null;
38
39          Departamento departamento = new Departamento();
40          departamento.setNomeDepartamento(nomeDepartamento);
41          departamento.setDescricaoDepartamento(descricaoDepartamento);
42          try {
43              GenericDAO dao = new DepartamentoDAOImpl();
44              if (dao.cadastrar(departamento)) {
45                  mensagem = "Departamento cadastrado com sucesso!";
46              } else {
47                  mensagem = "Problemas ao cadastrar Departamento!";
48              }
49              request.setAttribute("sucesso", mensagem);
50              request.getRequestDispatcher("cadastrardepartamento.jsp").forward(request, response);
51          } catch (Exception ex) {
52              System.out.println("Problemas ao cadastrar Departamento! Erro: " + ex.getMessage());
53              ex.printStackTrace();
54          }
55      }
56
57      // <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left
```

62. Verifique se as importações estão corretas

```
CadastrarDepartamento.java X
Código-Fonte Histórico
1  /*
2   * To change this template, choose Tools | Templates
3   * and open the template in the editor.
4   */
5   package br.com.projetoweb.controller;
6
7   import br.com.projetoweb.DAO.DepartamentoDAOImpl;
8   import br.com.projetoweb.DAO.GenericDAO;
9   import br.com.projetowebcrud.model.Departamento;
10  import java.io.IOException;
11  import java.io.PrintWriter;
12  import javax.servlet.ServletException;
13  import javax.servlet.http.HttpServlet;
14  import javax.servlet.http.HttpServletRequest;
15  import javax.servlet.http.HttpServletResponse;
16
17  /**
```

63. Feito isso, crie um novo **Servlet** para Listar as informações cadastradas

64. Criado o novo **Servlet**, implemente o código abaixo:

```
ListarDepartamento.java X
Código-Fonte Histórico
16  /**
17   *
18   * @author Igor Paulino
19   */
20  public class ListarDepartamento extends HttpServlet {
21
22      /**
23       * Processes requests for both HTTP
24       * <code>GET</code> and
25       * <code>POST</code> methods.
26       *
27       * @param request servlet request
28       * @param response servlet response
29       * @throws ServletException if a servlet-specific error occurs
30       * @throws IOException if an I/O error occurs
31       */
32      protected void processRequest(HttpServletRequest request, HttpServletResponse response)
33          throws ServletException, IOException {
34          try {
35              GenericDAO dao = new DepartamentoDAOImpl();
36              request.setAttribute("departamentos", dao.listar());
37              request.getRequestDispatcher("listardepartamento.jsp").forward(request, response);
38          } catch (Exception ex) {
39              System.out.println("Problemas na Controller ao listar Departamento! Erro:" + ex.getMessage());
40              ex.printStackTrace();
41          }
42      }
43
44      // <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left to
45      /**
```

65. Não se esqueça de alterar os nomes e o nome da página que irá listar.

66. Feito isso, iremos agora criar uma nova página **JSP** para mostrar para o usuário as informações cadastradas.

67. Abra a página criada e implemente o código abaixo:

```
listardepartamento.jsp x
Código-Fonte  Histórico
1 <!--
2 Document : listardepartamento
3 Created on : 24/03/2014, 22:10:15
4 Author : FC 18
5 -->
6
7 <%@page import="java.util.List"%>
8 <%@page import="br.com.projetoweborud.model.Departamento"%>
9 <%@page contentType="text/html" pageEncoding="UTF-8"%>
10 <!DOCTYPE html>
11 <html>
12 <head>
13 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
14 <title>Lista de Departamento</title>
15 </head>
16 <body>
17 <table border="1" align="center">
18 <thead>
19 <tr>
20 <th>ID</th>
21 <th>Nome</th>
22 <th>Descrição</th>
23 <th colspan="2">Editar</th>
24 </tr>
25 </thead>
26 <tbody>
27 <%
28 List<Departamento> departamentos = (List<Departamento>) request.getAttribute("departamentos");
29 for (Departamento departamento : departamentos) {
30 %>
31 <tr>
32 <td><%= departamento.getIdDepartamento() %></td>
33 <td><%= departamento.getNomeDepartamento() %></td>
34 <td><%= departamento.getDescricaoDepartamento() %></td>
35 <td><a href="ExcluirDepartamento?idDepartamento=<%=departamento.getIdDepartamento() %>">Excluir</a></td>
36 </tr>
37 <% } %>
38 </tbody>
39 </table>
40 </body>
41 </html>
42
```

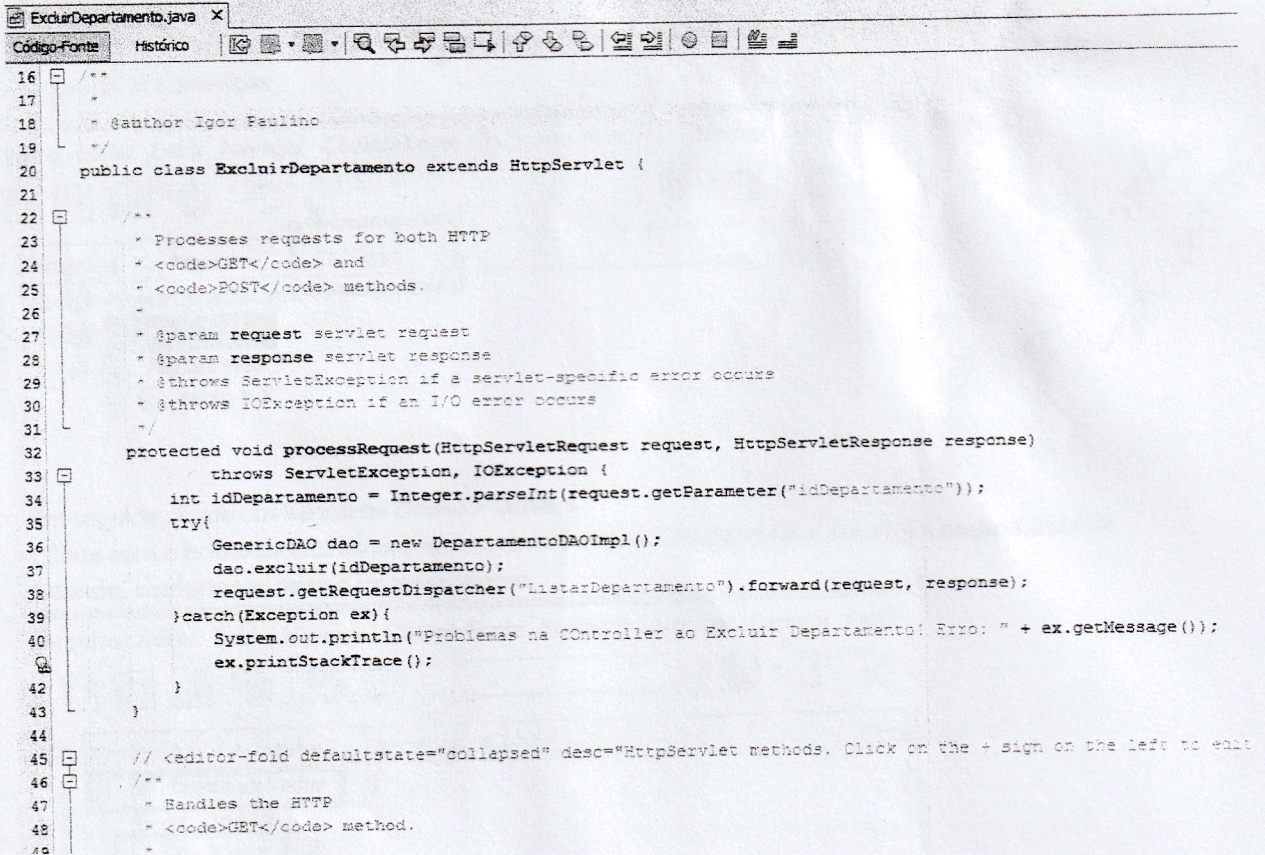
68. Não se esqueça de alterar os nomes e fazer as importações.

69. Pronto!

70. Agora iremos implementar o **Servlet** para Excluir o registro cadastrado.

71. Crie um novo **Servlet** no pacote **Controller**.

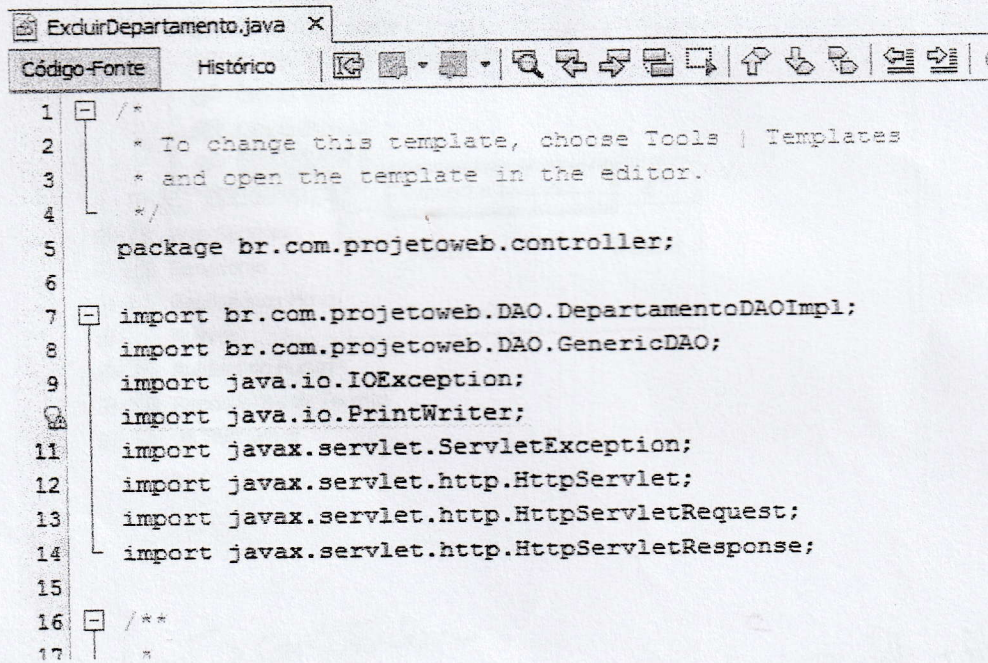
72. Dê um nome para esse Servlet. Ex: ExcluirAluno, ExcluirFuncionario.



```

16  /**
17   *
18   * @author Igor Paulino
19   */
20  public class ExcluirDepartamento extends HttpServlet {
21
22      /**
23       * Processes requests for both HTTP
24       * <code>GET</code> and
25       * <code>POST</code> methods.
26       *
27       * @param request servlet request
28       * @param response servlet response
29       * @throws ServletException if a servlet-specific error occurs
30       * @throws IOException if an I/O error occurs
31       */
32      protected void processRequest(HttpServletRequest request, HttpServletResponse response)
33          throws ServletException, IOException {
34          int idDepartamento = Integer.parseInt(request.getParameter("idDepartamento"));
35          try {
36              GenericDAO dao = new DepartamentoDAOImpl();
37              dao.excluir(idDepartamento);
38              request.getRequestDispatcher("ListarDepartamento").forward(request, response);
39          } catch (Exception ex) {
40              System.out.println("Problemas na Controller ao Excluir Departamento: Erro: " + ex.getMessage());
41              ex.printStackTrace();
42          }
43      }
44
45      // <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left to edit
46      /**
47       * Handles the HTTP
48       * <code>GET</code> method.
49       */
  
```

73. Verifique se as importações estão corretas



```

1  /**
2   * To change this template, choose Tools | Templates
3   * and open the template in the editor.
4   */
5  package br.com.projeto.web.controller;
6
7  import br.com.projeto.web.DAO.DepartamentoDAOImpl;
8  import br.com.projeto.web.DAO.GenericDAO;
9  import java.io.IOException;
10 import java.io.PrintWriter;
11 import javax.servlet.ServletException;
12 import javax.servlet.http.HttpServlet;
13 import javax.servlet.http.HttpServletRequest;
14 import javax.servlet.http.HttpServletResponse;
15
16 /**
17  *
  
```

74. E fim!